

# P3282R0

## Static Storage for C++ Concurrent `bounded_queue`

Published Proposal, 2024-05-19

**This version:**

<http://wg21.link/P3282R0>

**Author:**

[Detlef Vollmann](#)

**Audience:**

LEWG

**Project:**

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

**Source:**

[gitlab.com/cppz/papers/-/blob/main/source/P3282R0.bs](https://gitlab.com/cppz/papers/-/blob/main/source/P3282R0.bs)

---

## Abstract

Bounded concurrent queues are an important communication mechanism. However many embedded systems don't support dynamic memory allocation. This paper proposes to add a constructor to `bounded_queue` [P0260](#) that takes a pointer to the memory to be used for storage of queue elements.

## Table of Contents

- 1 Revision History
- 2 Introduction
- 3 Design

References

## § 1. Revision History

This paper is the initial revision.

## § 2. Introduction

*P0260* proposes `bounded_queue` as communication mechanism for concurrent systems. Such queues are often used in small embedded systems to communicate data from one task to another. *P0260* proposes a constructor for `bounded_queue` that takes the maximum number of elements that the queue can hold and optionally an allocator. The storage for the queue is allocated at construction time.

But many small embedded systems prefer to layout the complete memory usage at build time without support for dynamically allocated memory. This cannot be achieved even by providing a special allocator, as the amount of required memory is unknown. And even if it could be done by a special allocator, the programmatical overhead is not small.

To make statically allocated memory for `bounded_queue` possible, this paper proposes the addition of a constructor to `bounded_queue` that takes the number of elements of the queue and a pointer to memory to be used for the storage.

## § 3. Design

The additional constructor we propose here is pretty simple:

```
bounded_queue(size_t max_elems, void *storage);
```

The requirement is that `storage` points to enough memory for `max_elems`. However, how much memory is required for holding `max_elems` in a `bounded_queue` is generally not known.

So we propose another addition to `bounded_queue`:

```
static constexpr size_t required_size(size_t max_elems);
```

This function returns at compile time an upper bound to the size of the memory required to hold `max_elems` elements.

And we also need to know the required alignment:

```
static constexpr size_t required_alignment();
```

With this, a program could look like this:

```
typedef std::bounded_queue<uint32_t> MyQueueT;
alignas (MyQueueT::required_alignment)
std::array<std::byte, MyQueueT::required_size(8)> myQStorage;

MyQueueT myQ{8, myQStorage.data()};

int main()
{
    // start tasks and use myQ
}
```

## § References

### § Informative References

#### [P0260]

*P0260*. URL: <https://wg21.link/P0260>